

FLEET: From Logits Entropy to Enhanced Trajectories in Text Generation

Oleksii Streltsov* and Oleksandra Vitko

Department of Artificial Intelligence, Kharkiv National University of Radio Electronics, Kharkiv, Ukraine

Abstract

Solutions based on large language models (LLMs) often rely on temperature sampling to improve accuracy and stability by aggregating multiple samples from the completion distribution. However, this memoryless approach is inherently suboptimal: because it lacks awareness of prior generations and their evaluations, it produces an increasing proportion of semantically duplicate answers as more samples are drawn, leading to diminishing returns. To address this limitation, we introduce **FLEET**, a novel method that integrates a memory mechanism into the generation process. FLEET represents each generation as a sparse trajectory through states whose entropy exceeds a predefined threshold and uses these trajectories to infer per-token utility scores that adjust the logits. Benchmark evaluations demonstrate that FLEET achieves the same accuracy as the repeated sampling baseline, with a 3x speedup, and substantially improves accuracy on complex coding tasks (LiveCodeBench Pass@32 increases from 59.9% to 66.2%) under the same budget. Furthermore, in the greedy-decoding configuration evaluated here, the approach is deterministic and uses a single calibration LLM pass to derive its principal hyperparameters, requiring only minimal modifications to existing LLM pipelines.

1 Introduction

Large language models (LLMs) inherently operate under conditions of high uncertainty. Their capacity to function effectively in such environments stems from their robust predictive performance, which is largely maintained during autoregressive decoding (He and Su, 2024). However, the sequential nature of LLM generation implies that a localized failure at a pivotal step can induce cascading errors, ultimately derailing the entire reasoning trajectory. Consequently, there remains a pronounced disparity between the models’ high proficiency in isolated next-token prediction and their success rates in multi-step, autonomous problem-solving (agentic) tasks (Laban et al., 2025).

To address this limitation, recent research has increasingly focused on test-time scaling, a paradigm that leverages additional computational resources during inference to mitigate autoregressive failures (Zhang et al., 2025). Such methods include aggregating multiple model completions to identify the optimal or most frequent response (Zhou et al., 2025), building iterative self-refinement frameworks (Shinn et al., 2023) or training the models to intrinsically evaluate their intermediate

*Work was done while Oleksii Streltsov was a master’s student at Kharkiv National University of Radio Electronics.
Algorithm source code and experiments: <https://github.com/Alexiush/fleet>

outputs, enabling them to dynamically allocate supplementary inference compute based on task complexity (DeepSeek-AI et al., 2025).

In this work, we specifically focus on the paradigm of sampling multiple completions. First, prior literature demonstrates that sampling remains one of the most cost-effective and accessible test-time interventions (Snell et al., 2024). Consequently, methodological advancements in this domain are highly generalizable, yielding immediate performance benefits across virtually any decoder-only generative architecture (Yenduri et al., 2023). Second, in contrast to alternative methods that strictly depend on the intrinsic capabilities of the model, sampling affords fine-grained, inherently task-agnostic control over the generation trajectory. Finally, sampling mechanisms constitute a foundational component of the post-training alignment pipelines utilized in the development of modern large language models (Ouyang et al., 2022).

The conventional baseline for token selection in large language models is greedy decoding, which deterministically selects the candidate token associated with the highest conditional probability at each autoregressive step. However, this deterministic paradigm is inherently ill-suited for test-time scaling strategies, as it yields zero sample variance and produces identical completion trajectories across repeated invocations. To induce diversity into the generation process, stochastic temperature sampling is commonly employed. Its temperature-controlled distribution is related to the Boltzmann sampling used in early stochastic neural models (Ackley et al., 1985). Rather than selecting the distribution mode, temperature sampling scales the unnormalized model logits by a temperature parameter $T > 0$ prior to applying the softmax operator, thereby constructing a re-scaled probability distribution from which subsequent tokens are stochastically drawn. This mechanism enables the generation of distinct completion trajectories while maintaining probability mass aligned with the model’s underlying likelihood estimates (Brown et al., 2024):

$$x_{\text{next}} = \arg \max_{x \in \mathcal{V}} P(x) \quad (1)$$

$$P(x_i) = \frac{\exp(l_i)}{\sum_{x_j \in \mathcal{V}} \exp(l_j)} \quad (2)$$

$$P_T(x_i) = \frac{\exp(l_i/T)}{\sum_{x_j \in \mathcal{V}} \exp(l_j/T)} \quad (3)$$

We identify key structural limitations in conventional sampling approaches:

a) Sample Inefficiency Consider a generation containing a small subset of tokens that are crucial to task success, which we call branching points. Success requires selecting an appropriate branch at each such point. When the optimal token does not correspond to the highest-likelihood mode, standard sampling strategies continuously over-allocate probability mass to unviable candidates. As the frequency of these branching points increases, the joint probability of sampling a globally correct trajectory decays exponentially. Simply increasing the temperature parameter fails to resolve this issue. While a higher temperature elevates the likelihood of selecting non-modal optimal tokens, it uniformly inflates variance across all generation steps. This indiscriminate entropy injection destabilizes generation at otherwise stable steps, frequently degrading overall success rates. Consequently, temperature scaling does not constitute a principled solution to sample inefficiency; rather, it acts merely as a static control parameter to navigate the trade-off between exploration and precision.

b) Lack of Selective Exploration In any given state, only a narrow subset of the vocabulary represents valid or task-relevant continuations. Furthermore, a substantial fraction of autoregressive steps are predominantly syntactic or structural (e.g., punctuation, functional words, or deterministic code syntax), operating under low entropy. Ideally, stochastic sampling should be restricted to high-entropy decision points and semantically meaningful actions, rather than applied uniformly across all generation steps. To partially mitigate the inclusion of unviable tokens, heuristic logit truncation strategies are commonly integrated into the sampling pipeline:

- Top- k sampling, which restricts the candidate vocabulary to a fixed subset of k tokens with the highest conditional probabilities (Fan et al., 2018);
- Top- p (nucleus) sampling, which dynamically selects the minimal set of candidate tokens whose cumulative probability mass exceeds a threshold parameter p (Holtzman et al., 2019);
- Min- p sampling, which truncates candidate tokens whose conditional probability falls below a dynamic threshold scaled relative to the likelihood of the distribution mode $P_{\max}$ (Nguyen et al., 2024):

$$\mathcal{V}'_{top-k} = \{x \in \mathcal{V} \mid \text{rank}(P(x)) \leq k\} \quad (4)$$

$$\mathcal{V}'_{top-p} = \arg \min_{\mathcal{S} \subset \mathcal{V}} (|\mathcal{S}|) \quad \text{subject to} \quad \sum_{x \in \mathcal{S}} P(x) \geq p \quad (5)$$

$$\mathcal{V}'_{min-p} = \{x \in \mathcal{V} \mid P(x) \geq p \cdot P_{\max}\} \quad (6)$$

c) Low Robustness and Poor Cross-Task Generalization The concurrent deployment of temperature scaling and the aforementioned logit-truncation strategies yields a highly parameterized search space (encompassing T , k , p , and $p_{\min}$). Because these parameters lack a unified, principled theoretical foundation, their selection relies almost entirely on ad hoc empirical tuning. Consequently, optimal hyperparameter configurations exhibit significant sensitivity to prompt design, model architecture, and task complexity, failing to generalize across diverse downstream domains. As a result, practitioners rarely work with optimally tuned models and instead rely on domain-oriented configurations.

We are interested in an alternative strategy that is free from these limitations. We investigate existing logit-processing strategies beyond greedy decoding and temperature sampling and analyze how they mitigate the limitations described above. We find that current solutions can mitigate limitations (b) and (c), but not (a). We attribute this shortcoming to the fact that these sampling methods are unaware of the rewards associated with generated completions and therefore cannot distinguish between states in which they should explore and those in which they should exploit. As a solution to this problem, we propose a lightweight memory mechanism, an algorithm that uses it to sample better completions and evaluate this setup on two benchmarks with verifiable tasks.

In summary, our key contributions are as follows:

- We introduce Vector Disjoint Set Union (VectorDSU), an online data structure that maps hidden states into unified search states, preventing trajectory duplication and preserving state utility history.

- We propose FLEET, a memory-augmented, deterministic search paradigm that replaces memoryless temperature sampling with targeted exploration of the completion space.
- We provide an empirical analysis of FLEET performance comparing it to repeated sampling on mathematical and coding problems using both ground truth verifiers and reward models. We confirm that on these tasks our approach scales better as its accuracy is always higher under the same budget.

2 Related Work

One widely used alternative to greedy decoding is beam search, particularly in neural machine translation (Wu et al., 2016). At each decoding step, beam search expands the retained partial sequences with candidate next tokens and keeps a fixed number of the highest-scoring hypotheses. The final output is typically selected according to accumulated sequence log-probability, often with a length adjustment. This deterministic search procedure can improve sequence-level consistency without requiring an external completion evaluator. However, it does not resolve the limitations described above because its search remains guided by the model’s likelihood estimates; when high-likelihood continuations are incorrect, beam search may systematically favor them.

To address the inherent limitations of static decoding, several adaptive sampling algorithms have been introduced. These methods generally share the following strategy (Zhu et al., 2023) (Chang et al., 2025):

1. State Saliency Detection: An auxiliary scoring mechanism or heuristic evaluates the generation context at step i to identify critical or high-entropy decoding states (e.g., decision nodes characterized by high prediction uncertainty or task relevance).
2. Dynamic Parameter Modulation: Decoding hyperparameters (such as temperature T or truncation thresholds p , k) are adaptively re-scaled as a function of the detected state properties, thereby balancing exploration and exploitation on a step-by-step basis.

Formally, a representative adaptive policy that dynamically elevates the sampling temperature T_i when encountering a high-saliency state s_i can be expressed as:

$$T_i = \begin{cases} a = f(s_i) & \text{if } s_i \in \mathcal{S}', \\ b & \text{otherwise} \end{cases} \quad \text{where } a > b \quad (7)$$

While state-dependent modulation provides a principled mechanism to mitigate the lack of selective exploration (limitation b), it does not fully resolve the structural intricacies of test-time scaling. To address hyperparameter brittleness and poor generalization (limitation c), recent approaches have integrated learnable modules capable of governing dynamic parameter adjustments (Dang et al., 2026). Although these methods still rely on data-driven optimization, the tuning process is coupled directly with the objective of the target task, effectively bypassing the necessity for ad-hoc, task-agnostic manual searches.

Nevertheless, the fundamental vulnerability – sample inefficiency at critical decision nodes (limitation a) – persists. The persistence of this issue indicates that dynamically searching for an “optimal”

temperature is a fundamentally misaligned objective; scalar logit adjustments cannot selectively amplify specific valid tokens without concurrently inflating the variance of the entire distribution. Existing adaptive-temperature results are task-dependent, and their generality across broader domains remains unclear.

3 FLEET Algorithm

In the context of test-time scaling, the fundamental objective is rarely to faithfully approximate the model’s predictive distribution; rather, it is to isolate optimal, high-reward trajectories from within a vast hypothesis space. Standard temperature sampling, however, is inherently memoryless. It fails to leverage the evaluative feedback, derived from either verifiable task environments or learned preference models, that is usually used to evaluate the scaling process. Consequently, it has no concept of exploitation. Integrating a historical memory mechanism transforms this paradigm, enabling the decoding process to be steered via structured, search-like dynamics rather than blind stochasticity. By retaining evaluative information across generation iterations, such an approach can systematically navigate the combinatorial explosion of the token space, effectively bypassing the inefficiencies associated with temperature scaling. Consequently, we argue that resolving the aforementioned decoding limitations necessitates an algorithm designed to search the model’s completion space systematically, rather than merely sample from it.

To resolve the limitations outlined above, we introduce FLEET (From Logits Entropy to Enhanced Trajectories) – a memory-augmented sampling framework designed to systematically mitigate the structural inefficiencies of standard and adaptive decoding. FLEET employs an information-theoretic heuristic based on distribution entropy to dynamically detect high-uncertainty decoding states, identifying them as pivotal branching points. These states are subsequently indexed within a graph-like data structure, mapping local state representations to metadata that tracks historical completion trajectories traversing through them. Rather than relying on global scalar hyperparameter adjustments, FLEET utilizes this memory to evaluate candidate paths in a manner analogous to Monte Carlo Tree Search (MCTS) (Browne et al., 2012). By leveraging historical evaluative outcomes, the algorithm selectively applies targeted logit penalties to actions associated with suboptimal trajectories, redirecting probability mass toward more promising search branches. Furthermore, hyperparameter selection within FLEET is principled and data-driven, as opposed to the black-box optimization typically required by standard sampling pipelines.

Formally, the entropy heuristic is derived from the model’s unnormalized logit output, from which we compute two complementary information-theoretic metrics: conditional entropy H and varentropy V .

$$H(X) = - \sum_x p(x) \log p(x) \quad (8)$$

$$V(X) = \sum_x p(x) (\log p(x) + H(X))^2 \quad (9)$$

Employing a single uncertainty metric is insufficient, as conditional entropy and varentropy describe complementary aspects of the probability distribution’s shape:

- **Conditional Entropy (H):** Measures total distributional uncertainty. However, standard Shannon entropy is susceptible to false positives in the presence of semantically redundant tokens (e.g., synonym clusters or stylistic variations) and during structured reasoning steps (e.g., deterministic mathematical operations), where high entropy does not necessarily reflect true semantic divergence.
- **Varentropy (V):** Quantifies the variance (dispersion) of log-probabilities around the mean entropy. Varentropy remains relatively invariant under broad, semantically uniform synonym distributions, but exhibits pronounced spikes during multimodal decision steps where probability mass is split across distinct, non-equivalent candidate clusters.

For numerical scaling within the calibration pipeline, both metrics are normalized relative to the model’s embedding dimension.

Furthermore, empirical observations demonstrate that final-layer logits often fail to provide the most sensitive uncertainty signals due to late-stage probability smoothing. To capture sharper decision-making dynamics, we extract hidden representations $h^{(l)}$ from an intermediate layer $l < L$ and project them directly into the vocabulary space using the language model head W_U – an interpretability technique known as the Logit Lens ([nostalgebraist, 2020](#)):

$$\mathbf{l}_t^{(l)} = \mathbf{W}_U \mathbf{h}_t^{(l)} + \mathbf{b}_U \quad (10)$$

$$P^{(l)}(x_i) = \frac{\exp(l_{t,i}^{(l)})}{\sum_{x_j \in \mathcal{V}} \exp(l_{t,j}^{(l)})} \quad (11)$$

Because intermediate hidden states lie in a continuous vector space, identifying semantically equivalent decision nodes and aggregating historical trajectory statistics across generations constitutes a nontrivial clustering task. To dynamically map continuous representations to discrete equivalence classes, we introduce a special data structure termed Vector Disjoint Set Union (VectorDSU):

VectorDSU is motivated by an empirical property of representation alignment: above a calibrated threshold τ , high cosine similarity S_C between two hidden vectors is associated with bounded Kullback–Leibler (KL) divergence D_{KL} between their projected probability distributions P and Q over the vocabulary $\mathcal{V}$:

$$S_C(\mathbf{h}_1, \mathbf{h}_2) = \frac{\mathbf{h}_1 \cdot \mathbf{h}_2}{\|\mathbf{h}_1\| \|\mathbf{h}_2\|} \quad (12)$$

$$D_{KL}(P \parallel Q) = \sum_{x \in \mathcal{V}} P(x) \log \left(\frac{P(x)}{Q(x)} \right) \quad (13)$$

This alignment enables the robust mapping of continuous hidden states into discrete topological clusters $\mathcal{C}_k$, serving as anchor points to which trajectory metadata is attached:

$$S_C(\mathbf{h}_1, \mathbf{h}_2) \geq \tau \implies D_{KL}(P_1 \parallel P_2) \leq \epsilon, \quad \text{where } P_i = \text{softmax}(W\mathbf{h}_i + \mathbf{b}) \quad (14)$$

This operating criterion is selected empirically for the model and layer being calibrated. It is theoretically motivated by the use of normalized hidden states. The projection from intermediate

hidden states to output probabilities consists of a linear transformation (the language model head) followed by a softmax nonlinearity. Because the softmax function is Lipschitz-continuous and invariant to uniform scalar shifts, angular proximity in the latent continuous space intrinsically constrains the statistical divergence of the resulting output distributions. The exact tightness of this bound depends on the spectral norm of the LM head weights, which in practice are tightly bound by regularization and initialization schemes.

3.1 Online Hidden State to Search State Mapping via Vector Disjoint Set Union

The proposed online clustering mechanism is inspired by the disjoint-set data structure, commonly referred to as Disjoint Set Union (DSU) (Galil and Italiano, 1991). VectorDSU borrows DSU’s representative-centric organization, but it does not retain an exact union–find graph over all high-dimensional vectors. Instead, it maintains compact component representatives and the trajectory metadata associated with the cluster of vectors that resolve to it. Its operation follows three structural principles:

- **Canonical Representation:** Each component maintains a representative vector. Like in DSU vector that resolves to that representative is considered a member. Thus, once a state is assigned, its metadata resolves to that component even though the high-dimensional member vector need not be retained.
- **Virtual Connectivity:** Component membership is induced by the history of online assignments and merges, analogously to connectivity in DSU. It does not require every historical member to remain directly similar to the current representative.
- **Low-Memory Dynamic Union:** VectorDSU merges component identifiers, representatives, and metadata without storing every member vector. The representative may remain fixed or be updated as a running centroid, depending on the configured variant.

For a new hidden state $\mathbf{h}$, VectorDSU uses cosine similarity to choose an existing representative μ_i or to create a new component. This is an online assignment rule rather than a requirement that every historical member remain directly related to the representative. Retaining only representative vectors and compact component metadata substantially reduces memory use; representative lookup can additionally be accelerated through parallelization or spatial partitioning.

$$C(\mathbf{h}) = \arg \max_i S_C(\mathbf{h}, \mu_i) \quad \text{if} \quad \max_i S_C(\mathbf{h}, \mu_i) \geq \tau; \quad \text{otherwise create a new component.} \quad (15)$$

Once a state is mapped to a cluster, the cluster functions as a memory node, recording historical trajectory metadata. Specifically, each cluster aggregates transition tuples reflecting the generation dynamics. For a given autoregressive step originating in state $\mathcal{C}_t$, the stored metadata comprises:

- the action (token) executed a_t ;
- the state $\mathcal{C}_{t+1}$ reached upon executing action a_t ;

- the reward $R(s_t, a_t)$ accumulated along the trajectory path.

$$\mathcal{M}_t = \langle a_t, C_{t+1}, \bar{R}(s_t, a_t) \rangle \quad (16)$$

Because state cluster retrieval relies on intermediate vector representations, FLEET assumes access to the model’s internal activations and output probability distributions. This transparency enables the integration of Monte Carlo Tree Search (MCTS) to navigate the sequence space, specifically utilizing the predictor-guided Upper Confidence Bound applied to Trees (pUCT) formulation (Silver et al., 2017).

Standard UCT evaluates actions by balancing an empirical exploitation term $Q(s, a)$ – defined as the average observed reward for executing action a in state s – with an exploration bonus driven by relative visit counts and scaled by an exploration constant c . However, standard UCT assumes exhaustive local exploration, rendering it unsuited for vast vocabulary spaces. In contrast, pUCT incorporates an explicit prior policy $P(s, a)$ – naturally supplied by the language model’s unpenalized output probability distribution – to bias search toward semantically viable candidates. Additionally, it rescales the exploration dynamics to support online, non-exhaustive tree expansion:

$$UCT(s, a) = Q(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \quad (17)$$

$$pUCT(s, a) = Q(s, a) + c_{puct} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)} \quad (18)$$

Here, $N(s, a)$ is the number of times action a has been evaluated from state s , and $N(s) = \sum_a N(s, a)$ is the total visit count of that state.

Direct application of the standard pUCT formulation requires crucial modifications to accommodate the stochasticity and high dimensionality inherent to autoregressive language generation. Because a single token action a evaluated from state s may transition into a distribution of potential subsequent clusters, we evaluate action utility $Q(s, a)$ as the expectation of rewards across all known outcomes.

Furthermore, to mitigate the distortive effects of the model’s predictive priors, we compute $P(s, a)$ by applying the resampling temperature T_{resample} to the logits before softmax, following Equation (3), and then restrict the decision set to the k most probable tokens. Let $\mathcal{U}(s)$ be the tokens in this top- k set for which $N(s, a) = 0$. These candidates are aggregated into an “exploration action” whose prior mass is $\sum_{a \in \mathcal{U}(s)} P(s, a)$. When this action is selected, the configured decoding rule chooses a concrete token from $\mathcal{U}(s)$. Thus, unexplored tokens remain available as a joint alternative, whereas known suboptimal tokens can be penalized individually.

Finally, rather than explicitly dictating the final token selection or replacing standard decoding pipelines, FLEET intervenes as a soft constraining mechanism. We preserve the configured decoding strategy, but apply a logit penalty λ to empirically suboptimal tokens – specifically, those that have been decoded and evaluated, yet fail to maximize the updated pUCT objective:

$$\hat{l}_{t,a} = l_{t,a} - \lambda \cdot \mathbb{I} \left[N(s_t, a) > 0 \wedge a \neq \arg \max_{a'} pUCT(s_t, a') \right] \quad (19)$$

Furthermore, the decoupled VectorDSU memory permits state-action statistics or priors derived from one worker or task to be supplied to other FLEET workers. This provides an interface for cross-session or cross-domain transfer, although the benefit of such transfer is not evaluated in the present experiments. In addition to the language model’s localized predictive prior, the search policy can therefore incorporate a global domain prior when one is available.

In the absence of an external task-specific signal, explored actions receive a default multiplicative prior of 0.5, whereas the aggregated exploration action does not have one which can be interpreted as a prior of 1. This deliberately downscales evaluated actions relative to the unexplored candidate pool during early search; the value 0.5 is therefore an exploration bias rather than a learned domain probability.

$$\hat{l}_{t,a} = l_{t,a} - \lambda \cdot \mathbb{I} \left[N(s_t, a) > 0 \wedge a \neq \arg \max_{a'} \{pUCT(s_t, a') \cdot prior(s_t, a')\} \right] \quad (20)$$

In our experiments, as an experiment-specific heuristic, the logit penalty λ was dynamically set to the maximum logit while we evaluated greedy decoding from the FLEET-processed scores.

3.2 FLEET Search

Synthesizing the components detailed above, the complete operational workflow of the proposed method is formalized in Algorithm 1.

To sustain search-like exploration dynamics, FLEET maintains a fixed-capacity running buffer of recent entropy–varentropy pairs and measures the percentage that satisfies the joint trigger $H > \tau_H \wedge V > \tau_V$. When this observed threshold-hit rate falls below the target $\rho(i)$, the entropy and varentropy thresholds are relaxed using the empirical distribution in the buffer so that the hit rate approaches the target. The target percentage follows a cube-root schedule:

$$\rho(i) = \min(i^{1/3}, 100) \% \quad (21)$$

where i denotes the search iteration index. This monotonic schedule gradually increases the target fraction of token states processed by FLEET. The buffer-based update permits the search to continue expanding even when the initial thresholds become too selective for later iterations.

The FLEET architecture introduces several other operational advantages:

- FLEET admits coordinated parallelization: workers can reserve or be assigned distinct high-scoring branches in the shared search memory, reducing overlap between concurrently generated trajectories. Quantifying the resulting parallel speedup remains future work.
- Structural updates to VectorDSU and cluster-level penalty tables can be batched after each completed generation. During the next generation, only hidden-state lookup and application of the precomputed cluster penalties must occur online. This decoupled design ensures that the algorithm is fully compilable with highly optimized production inference backends that compile the computation graph.

Algorithm 1 FLEET: From Logits Entropy to Enhanced Trajectories

Require: Generator G , evaluator RM , prompt P , budget B

Require: Decoding rule `DECODE` (greedy in our experiments)

Require: Layer L , entropy thresholds (τ_H, τ_V) , DSU threshold τ_{dsu}

```
1: Initialize VectorDSU  $\mathcal{D}$  with threshold  $\tau_{dsu}$ 
2: Initialize FLEET worker  $W$  linked to  $\mathcal{D}$ 
3:  $i \leftarrow 1$ 
4: while  $W.\text{MaxReward}() < 1$  and  $B > 0$  do
5:    $Y \leftarrow \text{STARTGENERATION}(G, P)$  ▷ Reset the trajectory
6:    $W.\text{RESETTRAJECTORY}()$ 
7:    $t \leftarrow 0$ 
8:   while  $G$  is generating and  $\text{LASTTOKEN}(Y) \neq \langle \text{EOS} \rangle$  do
9:     Obtain hidden state  $\mathbf{h}_t^{(L)}$  from layer  $L$ 
10:    Compute  $H(P_t)$  and  $V(P_t)$  from  $\mathbf{h}_t^{(L)}$  via LogitLens
11:     $W.\text{UPDATEUNCERTAINTYBUFFER}(H(P_t), V(P_t))$ 
12:    if  $H(P_t) > \tau_H \wedge V(P_t) > \tau_V$  then
13:       $C_t \leftarrow \mathcal{D}.\text{FINDORCREATE}(\mathbf{h}_t^{(L)})$  ▷ Map to a discrete state
14:       $\mathbf{l}_t \leftarrow G.\text{GETLOGITS}()$ 
15:       $\hat{\mathbf{l}}_t \leftarrow \mathbf{l}_t - W.\text{CALCULATEPENALTY}(C_t)$ 
16:       $a_t \leftarrow \text{DECODE}(\hat{\mathbf{l}}_t)$  ▷ Greedy in our experiments
17:       $W.\text{REGISTERACTION}(C_t, a_t)$ 
18:    else
19:       $a_t \leftarrow \text{DECODE}(G.\text{GETLOGITS}())$ 
20:    end if
21:     $Y \leftarrow \text{APPEND}(Y, a_t)$ 
22:     $t \leftarrow t + 1$ 
23:  end while
24:   $R \leftarrow RM.\text{EVALUATE}(Y)$ 
25:   $W.\text{BACKPROPAGATEREWARD}(R)$  ▷ Update  $\bar{R}$  and  $N$  in  $\mathcal{D}$ 
26:   $\rho_i \leftarrow \min(i^{1/3}, 100)\%$ 
27:  if  $W.\text{THRESHOLDHITRATE}() < \rho_i$  then
28:     $(\tau_H, \tau_V) \leftarrow W.\text{ADJUSTTHRESHOLDS}(\rho_i)$ 
29:  end if
30:   $B \leftarrow B - 1$ 
31:   $i \leftarrow i + 1$ 
32: end while
```

- Because the search operates as a soft constraint mechanism – guiding generation through dynamic logit penalization rather than replacing the configured decoder – it does not require changes to downstream evaluation beyond receiving a different generated sequence.
- VectorDSU constructs token-level attributed trajectory data that records which probable actions were attempted, their empirical rewards, and how they compare with alternatives. This information is richer than a single sequence-level correctness label and may be useful for downstream Supervised Fine-Tuning (SFT) or Reinforcement Learning (RL), although its training value is not evaluated here.

We also acknowledge several limitations in the current implementation:

- Unlike standard sampling techniques that operate entirely post-hoc on the generated logits, our approach currently requires direct patching of the underlying model architecture. While it is theoretically possible to apply this method exclusively to the output logits, we expect that it will require significantly more memory and operations, possibly with degraded performance.
- Our method can struggle when the model enters certain specific cognitive states, most notably those associated with induction or deep reflection. In inductive scenarios, defaulting to deterministic selection often mitigates the issue. However, for reflective states – which are increasingly central to the performance of “thinking” or reasoning-focused models – deterministic selection is actively detrimental. Consequently, our current method may not synergize well with advanced reasoning models that rely heavily on these internal reflective processes.
- Finally, we observe that the generated text can occasionally exhibit high levels of noise, where structurally less important tokens are degraded into unintelligible sequences (gibberish). While this phenomenon of sampling noise is not unique to our specific method, it remains a practical limitation. A secondary, lightweight language model may mitigate primarily syntactic or formatting corruption by rewriting the generated output, but this mitigation is not evaluated in the present experiments.

3.3 Hyperparameter Initialization

FLEET requires the following hyperparameters to be set:

- L : The intermediate layer index from which continuous hidden states are extracted.
- τ_H, τ_V : The baseline trigger thresholds for conditional entropy and varentropy, respectively.
- T_{resample}, k : The resampling temperature and the top- k truncation bound utilized during prior policy evaluation.
- τ_{dsu} : The critical cosine similarity threshold governing equivalence class formation within the VectorDSU.

Although FLEET lacks universal “default” settings – a departure from conventional stochastic sampling methods – its initialization process circumvents the need for exhaustive, iterative grid searches over validation datasets. Because each hyperparameter maps directly to a distinct mechanistic function within the search architecture, the consequences of parameter adjustment are comparatively interpretable. In our experiments, the FLEET configuration was derived from a single calibration pass, whereas the temperature-sampling baseline required repeated generations during temperature parameter optimization through Bayesian search.

To systematically identify the optimal intermediate layer for hidden state extraction, we introduce a composite scoring mechanism evaluated over a calibration prompt. For each candidate layer, this heuristic evaluates three primary criteria:

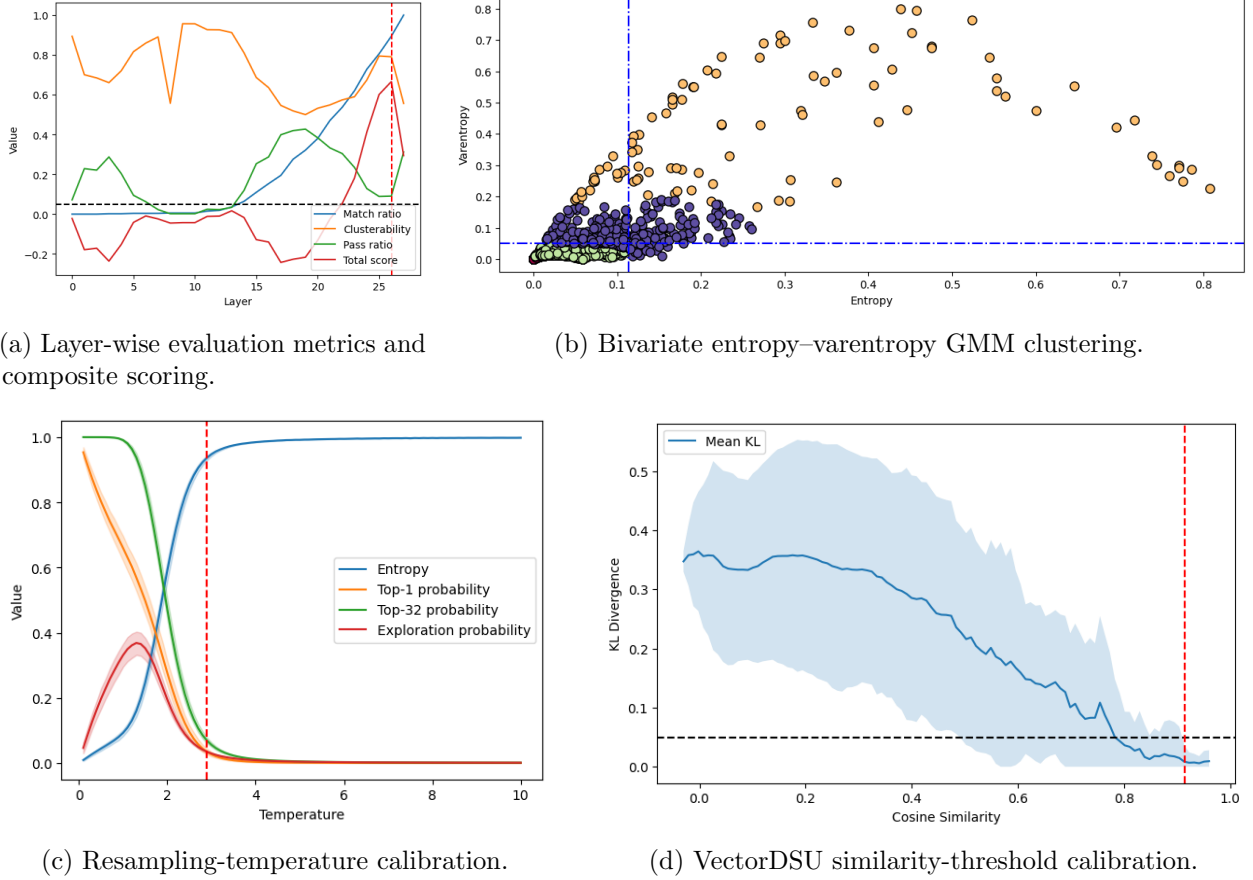


Figure 3.1: Illustrative diagnostics from an example calibration dataset used to refine the FLEET hyperparameter-selection pipeline for Llama 3.2-3B. Task-specific configurations are reported in Table 4.1.

- Predictive Alignment (R_{match}): The proportion of intermediate token representations – projected to the vocabulary space via the logit lens technique – that identically match the model’s final output sequence;
- Clustering Separability ($S_{\text{silhouette}}$): The topological separability of conditional entropy and varentropy into distinct bimodal distributions, quantified by the Silhouette coefficient;
- Distributional Deviation ($\Delta_{\text{expected}}(L)$): The divergence between the empirical fraction of states exceeding the derived decision threshold and the target theoretical expectation.

Synthesizing these metrics, the final layer-selection score S is computed as:

$$S(L) = R_{\text{match}}(L) \cdot S_{\text{silhouette}}(L) - \Delta_{\text{expected}}(L) \quad (22)$$

The complete calibration workflow is summarized in Figure 3.1.

Figure 3.1a shows the distinct structural trade-offs observed across the intermediate layers of Llama 3.2-3B. As expected, the logit lens match ratio scales monotonically toward the terminal layers,

reflecting progressive convergence onto the output vocabulary. However, an inverse relationship emerges between match ratio and topological clusterability (Silhouette score). Consequently, the final transformer layers prove suboptimal under our selection criteria due to severe degradation in entropy-varentropy separability.

Upon selecting the optimal target layer, we construct an expanded calibration dataset of continuous hidden states and corresponding output logits sampled across multiple prompts.

To parameterize decision boundaries within the joint two-dimensional entropy-varentropy space (H, V) , initial thresholds are rederived via bivariate clustering using a Gaussian Mixture Model (GMM) (Bishop, 2006). GMM density estimation effectively captures complex multi-modal cluster geometries across varying spatial densities. Contrary to our initial hypothesis of a simple quadrant-based partition (delineating high/low regimes of entropy and varentropy), the empirical joint distribution exhibits a cascading cluster topology. Given that layer selection favors independent metric clusterability, extracting decision thresholds from the primary mixture components refines the univariate boundaries by setting them equal to marginal points of corresponding axes on a joint 2D decision frontier.

Figure 3.1b visualizes the two-dimensional Gaussian Mixture Model clustering applied to the empirical distribution of state entropies and varentropies. The projection captures distinct density gradients that robustly delineate discrete topological regimes. The analytically derived decision thresholds, which partition these states, are demarcated by the blue dash-dotted lines.

Beyond spatial thresholds, the resampling temperature T_{resample} and the truncation bound k are explicitly parameterized to govern the underlying search dynamics. The candidate window size k is constrained by the expected computational search budget. Subsequently, to impose a structural bias toward early-stage exploration, the temperature scalar T_{resample} is dynamically calibrated to a critical transition point. Specifically, T_{resample} is annealed to the minimal value where the cumulative probability mass of the exploratory candidate tail (i.e., the $k - 1$ subordinate tokens) strictly outweighs the exploitative probability mass of the single maximum-likelihood candidate (top-1).

Figure 3.1c illustrates the temperature-dependent evolution of cumulative probability masses across distinct token partitions: the maximum-likelihood candidate (exploitation, top-1), the truncated candidate window (top- k , parameterized here as $k = 32$), and the exploratory tail (top- $k - \text{top-1}$). The optimal temperature scalar is empirically identified at the critical intersection point where the exploratory mass strictly surpasses the exploitative mass (observed at $T \approx 2.9$). Furthermore, the plot overlays the corresponding normalized entropy, providing a quantitative measure of how close the annealed distribution approaches a uniform noise prior, thereby guaranteeing that semantic coherence is maintained.

The VectorDSU similarity threshold (τ_{dsu}) was calibrated by leveraging the spatial cosine similarity-to-divergence correspondence established during the structural formulation. Figure 3.1d plots the upper confidence bound of this transition divergence (mean plus two standard deviations, $\mu + 2\sigma$) as a function of increasing state proximity. The final operational threshold τ_{dsu} (demarcated in red) is formally defined as the minimal similarity value beyond which the normalized divergence is strictly bounded below an error tolerance of 0.05 (demarcated in black).

4 Experiments

We empirically validate the proposed FLEET algorithm across two standardized benchmarks characterized by objectively verifiable reward functions: LiveCodeBench (Jain et al., 2024) for algorithmic code generation, and GSM8K (Cobbe et al., 2021) for mathematical reasoning. We benchmark the performance of FLEET against standard stochastic temperature sampling across two distinct evaluative paradigms:

- **Ground-Truth Verification:** This setting establishes an empirical upper bound by assuming access to deterministic, ground-truth-guided feedback. FLEET is supplied with exact programmatic rewards upon sequence completion: a scalar value of 1.0 for absolute correctness, or a continuous partial reward $r \in [0, 1)$ proportional to the execution correctness or error severity, as evaluated by the benchmark’s execution environment. Specifically, the LiveCodeBench reward maps directly to the fraction of passed unit tests, whereas the GSM8K reward constitutes a sparse, binary signal ($r \in \{0, 1\}$).
- **Outcome Reward Model (ORM) Guidance:** To simulate a realistic deployment environment without ground-truth access, search trajectories are steered using a surrogate ORM. Under this regime, the ORM acts as the terminal evaluator, supplying FLEET with continuous reward signals normalized to the same $[0, 1]$ range as the ground-truth verifier. The underlying scores are derived from parameterized preference distributions learned via pairwise preference optimization.

To mitigate data contamination, the LiveCodeBench evaluation is confined to 222 “easy” tasks sourced from programming contests held after the cutoff date reported as December 2023; the exact filtering rule and task list are provided in the experiment repository. For the GSM8K evaluation, two demonstrative examples are randomly selected offline from the official training split for each question and then fixed for the corresponding evaluation run.

For every problem, we generate $n = 32$ trajectories and report the complete scaling curve for $1 \leq k \leq n$. Under ground-truth evaluation, temperature sampling uses the standard combinatorial Pass@ k estimator (Chen et al., 2021). If c of the n sampled trajectories are correct, then

$$\widehat{\text{Pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}. \quad (23)$$

Because the evaluated FLEET configuration uses deterministic greedy decoding, its prefix-based value at k is 1 if any of the first k trajectories is correct and 0 otherwise. These per-problem values are then averaged across the benchmark.

Consequently, metric calculations are adapted to align with the underlying search mechanisms, particularly when deploying an Outcome Reward Model (ORM) for solution selection:

- **Deterministic Selection (FLEET):** Let $y_i \in \{0, 1\}$ be the correctness of trajectory i and r_i its ORM score. For the deterministic FLEET prefix, ORM-selected accuracy at k is y_{j_k} , where $j_k = \arg \max_{1 \leq i \leq k} r_i$.

- Stochastic Selection (Baseline Temperature Sampling): For the n sampled trajectories, ORM-selected accuracy at k is the average over all size- k subsets S :

$$\hat{A}_k^{\text{ORM}} = \binom{n}{k}^{-1} \sum_{\substack{S \subseteq \{1, \dots, n\} \\ |S|=k}} y_{\arg \max_{i \in S} r_i}. \quad (24)$$

This estimates the probability that the ORM’s highest-scoring candidate is correct when k candidates are selected from the sampled pool.

We employ Llama 3.2-3B as the core base language model because it provides a practical balance among inference throughput, computational requirements, and task performance. The present evaluation is limited to this dense autoregressive architecture; applying FLEET to Mixture-of-Experts (MoE), multimodal, or reflection-heavy reasoning architectures may require architecture-specific calibration.

To ensure structural and distributional representation alignment across the pipeline, we deploy Skywork-Reward-V2 (Liu et al., 2025) – which is also fine-tuned on the Llama 3.2-3B architecture – as our Outcome Reward Model (ORM).

Hyperparameters for FLEET were selected using the automated layer and threshold calibration protocol detailed in Section 3.3. To ensure a rigorous baseline comparison, the sampling temperature T for standard stochastic decoding was independently optimized via Bayesian search over an isolated split of the target benchmarks.

The complete experimental infrastructure was implemented using the Hugging Face Transformers library (Wolf et al., 2019) combined with nnsight (Fiotto-Kaufman et al., 2024) for low-overhead internal-state activation inspection and dynamic logit interventions during the forward pass. Exact checkpoint identifiers, software versions, prompts, and experiment configurations are provided in the linked repository.

All generations are limited to 1,024 new tokens. A sequence that reaches this limit without producing an end-of-sequence token is evaluated exactly as generated and is treated as unfinished; no additional completion or repair step is applied during evaluation. Baseline temperature sampling uses sampling-enabled decoding with the task-specific temperature reported in Table 4.1. FLEET uses greedy decoding and applies its penalties to the model’s raw, unscaled output logits before token selection. The system-prompt structure was adapted from the prompting setup used by Laban et al. (2025) and then specialized for the respective coding and mathematical-reasoning tasks. The complete prompts and the GSM8K few-shot construction routine are reproduced in Appendix A.

Task	Method	Configuration
GSM8K	Temperature sampling	$T = 0.6$
	FLEET	$L = 26, \tau_H = 0.11, \tau_V = 0.05, k = 32, \tau_{\text{dsu}} = 0.9, T_{\text{resample}} = 3.1$
LiveCodeBench	Temperature sampling	$T = 0.8$
	FLEET	$L = 27, \tau_H = 0.09, \tau_V = 0.03, k = 32, \tau_{\text{dsu}} = 0.9, T_{\text{resample}} = 1.0$

Table 4.1: Task-specific hyperparameter configurations for baseline temperature sampling and FLEET. L is the LogitLens layer; τ_H and τ_V are entropy and varentropy thresholds; k is the pUCT candidate-window size; τ_{dsu} is the VectorDSU similarity threshold; and T_{resample} is the temperature used to compute the pUCT prior.

4.1 Empirical Results

4.1.1 Ground-Truth Verification

Table 4.2 summarizes the comparative Pass@32 performance of the FLEET algorithm and the baseline temperature sampling method under the ground-truth verification regime. In this setting, the search process receives ground-truth feedback upon the completion of each trajectory.

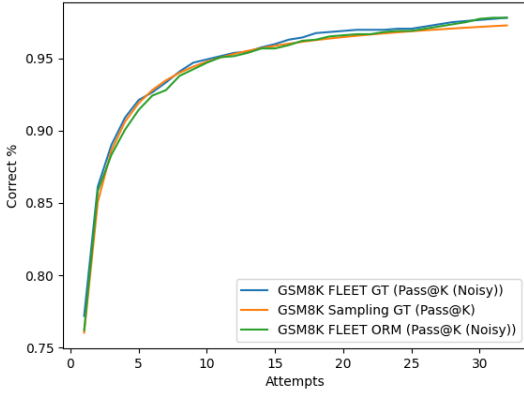
The empirical results demonstrate that FLEET outperforms standard stochastic sampling across both evaluation domains in this evaluation. The most substantial gains are observed on the LiveCodeBench programming benchmark, where FLEET achieves a 6.31-percentage-point absolute increase in Pass@32 accuracy, resolving 14 additional algorithmic tasks compared to the baseline.

For the GSM8K mathematical reasoning dataset, baseline temperature sampling exhibits performance saturation, successfully resolving 97.27% of the problem set. Despite this ceiling effect, the FLEET framework maintains a positive advantage in the reported run by successfully decoding 7 additional mathematical tasks that the baseline failed to resolve within the same candidate budget.

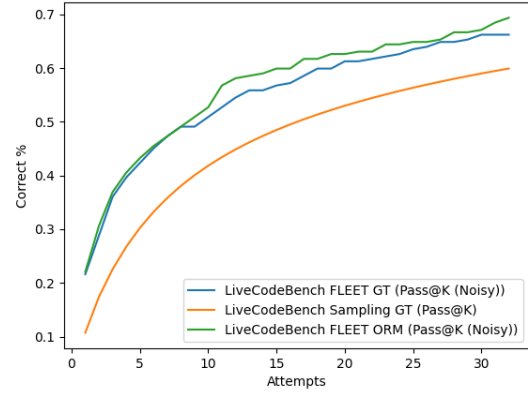
Condition	Accuracy (%)	Problems solved
GSM8K		
Baseline (temperature sampling)	97.27	1283
FLEET	97.80 (+0.53)	1290 (+7)
LiveCodeBench		
Baseline (temperature sampling)	59.90	133
FLEET	66.21 (+6.31)	147 (+14)

Table 4.2: Pass@32 under ground-truth verification. Parenthesized values are deltas from the baseline within each task; accuracy deltas are percentage-point differences.

Figure 4.1a illustrates the scaling efficiency of the decoding strategies as candidate budgets increase on the GSM8K dataset. By systematically enforcing orthogonal search paths via the VectorDSU mechanism, FLEET avoids redundant trajectory exploration and demonstrates better scaling



(a) GSM8K.



(b) LiveCodeBench.

Figure 4.1: Pass@ k accuracy scaling with candidate budget under ground-truth evaluation. Curves labeled “FLEET ORM” show ground-truth evaluation of candidates generated with ORM feedback.

dynamics.

Figure 4.1b illustrates the comparative scaling behavior of FLEET and baseline temperature sampling across increasing candidate budgets on the LiveCodeBench dataset. Unlike GSM8K where the model shows high base accuracy, programming proves to be more challenging.

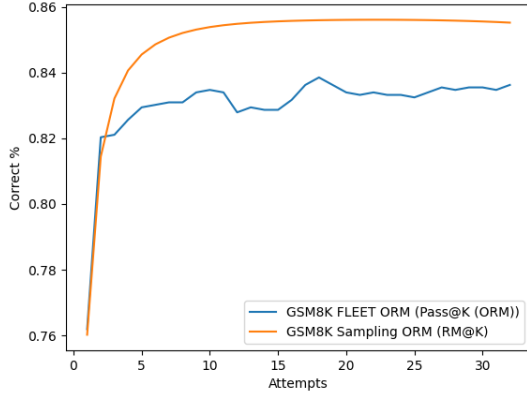
In contrast to temperature sampling, FLEET demonstrates a steeper and more robust scaling trajectory.

4.1.2 Outcome Reward Model (ORM) Guidance

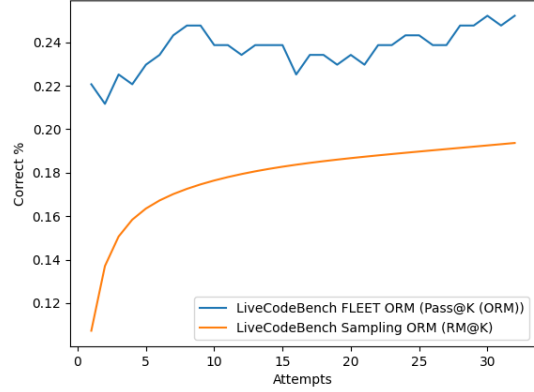
Table 4.3 summarizes the comparative Pass@32 performance under this ORM-guided regime, alongside an auxiliary evaluation – denoted as FLEET (GT-evaluated) – that measures the true underlying correctness of the trajectories generated by FLEET when judged by the ground-truth verifier.

LiveCodeBench: Under ORM selection, FLEET outperforms standard temperature sampling, raising the solution rate from 19.36% (43 problems) to 25.22% (56 problems). However, comparing the ORM-selected performance (0.2522) against FLEET’s ground-truth-evaluated candidate-pool accuracy (0.6937) exposes a substantial reward-ranking gap. This indicates that while the search framework successfully discovers correct algorithmic solutions within its candidate pool, the surrogate reward model struggles to reliably rank them above incorrect alternatives. It also shows that while ORM is not a reliable verifier for LiveCodeBench, its feedback does not make FLEET diverge into reward hacking: the GT-evaluated score does not plateau earlier than the GT-guided score.

GSM8K: On the mathematical reasoning benchmark, baseline temperature sampling marginally outperforms FLEET under ORM selection (0.8544 versus 0.8362). We hypothesize that this occurs because the reward model uses indirect cues for answer ranking – a byproduct of its conditioning on preference data rather than ground-truth approximation. Crucially, however, the FLEET (GT-evaluated) metric achieves 0.9780 (1290 solved problems), identical to its standalone ground-truth



(a) GSM8K.



(b) LiveCodeBench.

Figure 4.2: Pass@ k accuracy scaling with candidate budget under ORM guidance.

performance.

Condition	Accuracy (%)	Problems solved
GSM8K		
Baseline (temperature sampling, ORM-selected)	85.44	1127
FLEET, ORM-selected	83.62 (-1.82)	1103 (-24)
FLEET, ground-truth-evaluated	97.80 (+12.36)	1290 (+163)
LiveCodeBench		
Baseline (temperature sampling, ORM-selected)	19.36	43
FLEET, ORM-selected	25.22 (+5.86)	56 (+13)
FLEET, ground-truth-evaluated	69.37 (+50.01)	154 (+111)

Table 4.3: Pass@32 using an ORM for final candidate selection, alongside ground-truth evaluation of FLEET-generated trajectories. Parenthesized values are deltas from the baseline within each task.

Figure 4.2a illustrates the comparative scaling behavior of baseline temperature sampling and FLEET when candidate selection is mediated by an Outcome Reward Model (Skywork-Reward-V2) across increasing candidate budgets on the GSM8K dataset. In contrast to the ground-truth-guided setting, the ORM-evaluated scaling curves reveal a performance plateau and a widening gap relative to true underlying generation capability. This divergence underscores the susceptibility of surrogate reward models to ranking errors when evaluating structurally diverse trajectories. Specifically, because FLEET induces exploration through entropy-varentropy triggers and VectorDSU, the resulting paths may diverge from standard stylistic patterns preferred by the proxy reward model, leading to suboptimal candidate selection at higher candidate budgets.

Figure 4.2b illustrates the scaling dynamics of FLEET versus baseline temperature sampling on LiveCodeBench when candidate selection is mediated by the Skywork-Reward-V2 outcome reward model. Unlike the performance compression observed in mathematical reasoning, FLEET maintains a sustained and widening performance advantage over stochastic sampling as the candidate budget expands. This suggests that structured, entropy-guided exploration degrades less under imperfect

surrogate reward guidance on the more complex task evaluated here.

4.2 Discussion and Synthesis of Experimental Findings

A holistic evaluation of the empirical results highlights distinct operational trade-offs and behavioral patterns within the FLEET framework:

- **Advantage Under Ground-Truth Verification:** When supplied with deterministic, ground-truth feedback, FLEET outperforms standard stochastic temperature sampling across both evaluated domains. The performance delta is modest on GSM8K due to the already high ceiling, but pronounced on LiveCodeBench, where FLEET resolves 14 additional programming tasks out of 222 total (~ 6.3 percentage points). This result is consistent with FLEET suppressing redundant sampling and directing the candidate budget toward distinct reasoning paths.
- **The Surrogate Reward Bottleneck:** Coupling search frameworks with an Outcome Reward Model introduces notable performance shifts. Under ORM selection, baseline sampling marginally surpasses FLEET on GSM8K, whereas FLEET retains its superiority on LiveCodeBench, securing a lead of 13 tasks over the baseline despite overall performance degradation across both methods. A key advantage of integrating an ORM within the search loop is continuous reward shaping. While GSM8K’s standard verifier provides sparse binary signals, utilizing an ORM populates FLEET’s internal memory with smooth, continuous score trajectories. In hybrid configurations – where search dynamics leverage dense ORM score memory but final evaluation uses ground truth – FLEET resolves an additional 7 LiveCodeBench tasks within the same candidate budget. This suggests that surrogate models can provide useful dense guidance for internal search routing even when their terminal ranking accuracy is suboptimal.

5 Conclusion

In this work, we introduced FLEET, a test-time search framework that augments repeated generation with deterministic, state-aware trajectory exploration in its greedy-decoding configuration. By leveraging a dedicated structural memory (VectorDSU) to organize intermediate generation paths and dynamically adjusting exploration via entropy–varentropy trigger regimes, FLEET enhances compute-optimal test-time scaling.

Evaluation across mathematical reasoning and code-generation benchmarks demonstrates the potential of the approach. While maintaining competitive performance under high-baseline saturation on GSM8K, ground-truth-guided FLEET improves LiveCodeBench Pass@32 accuracy from 59.9% to 66.2%. When ORM feedback guides the search and the resulting candidate pool is evaluated using ground truth, accuracy reaches 69.4%, indicating additional candidate-generation potential that the ORM does not reliably recover during final selection. FLEET therefore provides a practical approach for augmenting large language models with structured completion-space search.

References

- David H. Ackley, Geoffrey E. Hinton, and Terrence J. Sejnowski. A learning algorithm for Boltzmann Machines. *Cognitive Science*, 9(1):147–169, mar 1985. ISSN 0364-0213. doi: 10.1016/s0364-0213(85)80012-4.
- Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer, 2006. ISBN 0387310732.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. <https://arxiv.org/abs/2407.21787v3>, jul 2024.
- Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfschagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, mar 2012. ISSN 1943-068X. doi: 10.1109/tciaig.2012.2186810.
- Haw-Shiuan Chang, Nanyun Peng, Mohit Bansal, Anil Ramakrishna, and Tagyoung Chung. REAL sampling: Boosting factuality and diversity of open-ended generation by extrapolating the entropy of an infinitely large LM. *Transactions of the Association for Computational Linguistics*, 13: 760–783, jul 2025. ISSN 2307-387X. doi: 10.1162/tacl.a_00757.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. <https://arxiv.org/abs/2107.03374v2>, jul 2021.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. <https://arxiv.org/abs/2110.14168v2>, oct 2021.
- Haoran Dang, Cuiling Lan, Hai Wan, Xibin Zhao, and Yan Lu. Temperature as a meta-policy: Adaptive temperature in LLM reinforcement learning. <https://arxiv.org/abs/2602.11779v1>, feb 2026.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang

- Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanxia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. <https://arxiv.org/abs/2501.12948v2>, jan 2025.
- Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. <https://arxiv.org/abs/1805.04833v1>, may 2018.
- Jaden Fiotto-Kaufman, Alexander R. Loftus, Eric Todd, Jannik Brinkmann, Koyena Pal, Dmitrii Troitskii, Michael Ripa, Adam Belfki, Can Rager, Caden Juang, Aaron Mueller, Samuel Marks, Arnab Sen Sharma, Francesca Lucchetti, Nikhil Prakash, Carla Brodley, Arjun Guha, Jonathan Bell, Byron C. Wallace, and David Bau. NNSight and NDIF: Democratizing access to open-weight foundation model internals. <https://arxiv.org/abs/2407.14561v4>, jul 2024.
- Zvi Galil and Giuseppe F. Italiano. Data structures and algorithms for disjoint set union problems. *ACM Computing Surveys*, 23(3):319–344, sep 1991. ISSN 0360-0300. doi: 10.1145/116873.116878.
- Hangfeng He and Weijie J. Su. A law of next-token prediction in large language models. <https://arxiv.org/abs/2408.13442v3>, aug 2024.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. <https://arxiv.org/abs/1904.09751v2>, apr 2019.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. <https://arxiv.org/abs/2403.07974v2>, mar 2024.
- Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. Llms get lost in multi-turn conversation. <https://arxiv.org/abs/2505.06120v1>, may 2025.
- Chris Yuhao Liu, Liang Zeng, Yuzhen Xiao, Jujie He, Jiakai Liu, Chaojie Wang, Rui Yan, Wei Shen, Fuxiang Zhang, Jiacheng Xu, Yang Liu, and Yahui Zhou. Skywork-Reward-V2: Scaling preference data curation via human-AI synergy. <https://arxiv.org/abs/2507.01352v3>, jul 2025.

- Minh Nhat Nguyen, Andrew Baker, Clement Neo, Allen Roush, Andreas Kirsch, and Ravid Schwartz-Ziv. Turning up the heat: Min-p sampling for creative and coherent LLM outputs. <https://arxiv.org/abs/2407.01082v8>, jul 2024.
- nostalgebraist. interpreting GPT: the logit lens. <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>, 2020.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. <https://arxiv.org/abs/2203.02155v1>, mar 2022.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. <https://arxiv.org/abs/2303.11366v4>, mar 2023.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. <https://arxiv.org/abs/1712.01815v1>, dec 2017.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. <https://arxiv.org/abs/2408.03314v1>, aug 2024.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. HuggingFace’s transformers: State-of-the-art natural language processing. <https://arxiv.org/abs/1910.03771v5>, oct 2019.
- Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google’s neural machine translation system: Bridging the gap between human and machine translation. <https://arxiv.org/abs/1609.08144v2>, sep 2016.
- Gokul Yenduri, Ramalingam M, Chemmalar Selvi G, Supriya Y, Gautam Srivastava, Praveen Kumar Reddy Maddikunta, Deepti Raj G, Rutvij H Jhaveri, Prabadevi B, Weizheng Wang, Athanasios V. Vasilakos, and Thippa Reddy Gadekallu. Generative pre-trained transformer: A comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions. <https://arxiv.org/abs/2305.10435v2>, may 2023.
- Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Wenyue Hua, Haolun Wu, Zhihan Guo, Yufei Wang, Niklas Muennighoff, Irwin King, Xue Liu, and Chen Ma. A survey on test-time scaling in large language models: What, how, where, and how well? <https://arxiv.org/abs/2503.24235v3>, mar 2025.

Zhi Zhou, Yuhao Tan, Zenan Li, Yuan Yao, Lan-Zhe Guo, Yu-Feng Li, and Xiaoxing Ma. A theoretical study on bridging internal probability and self-consistency for LLM reasoning. <https://arxiv.org/abs/2510.15444v1>, oct 2025.

Yuqi Zhu, Jia Li, Ge Li, YunFei Zhao, Jia Li, Zhi Jin, and Hong Mei. Hot or cold? adaptive temperature sampling for code generation with large language models. <https://arxiv.org/abs/2309.02772v3>, sep 2023.

A Prompts and Generation Configuration

A.1 LiveCodeBench system prompt

The LiveCodeBench problem specification is supplied as the user message following this system prompt:

```
"""
You are an expert Python programmer. You will be given a question (problem specification)
and will generate a correct Python program that matches the specification and passes
all tests.

Format:
- [Standalone] Make sure that your answer consists of only one Python function at the top
level. Do not wrap with a class or split into multiple functions.
"""
```

A.2 GSM8K system prompt

```
"""
You are a helpful assistant. Your task is to help solving simple math problems. Try to
break the problem into substeps, so it is transparent how
you have arrived to the final solution, just like in example QA pairs.

Format:
- Final answer should be a number, not an expression and is always the final line of the
solution, preceded by ####.
"""
```

A.3 GSM8K task and few-shot prompt construction

The following routine constructs the test tasks and independently selects two training examples offline for each question. The constructed prompts are then fixed for the corresponding evaluation run.

```
repo_id = "openai/gsm8k"
gsm_dataset = load_dataset(repo_id, 'main', split='test')
gsm_few_shots = load_dataset(repo_id, 'main', split='train')

math_tasks = [item for item in gsm_dataset]

def clean_answer(question):
    return re.sub(r"<<.*>>", "", question)

def get_few_shot_prompt(examples_count=2):
    random_examples = []

    for _ in range(examples_count):
        example_id = random.randint(1, len(gsm_few_shots)) - 1
```

```

        random_examples.append(example_id)

few_shot_items = gsm_few_shots.select(random_examples)

few_shot_pieces = []
for f in few_shot_items:
    few_shot_prompt = f"Question: {f['question']}\nAnswer: {clean_answer(f['answer'])}\n\n"
    few_shot_pieces.append(few_shot_prompt)

few_shot_prompt = "".join(few_shot_pieces)

return few_shot_prompt

for i, item in enumerate(math_tasks):
    item['task_id'] = f'gsm8k_{i}'
    item['source'] = 'gsm8k'
    item['prompt'] = get_few_shot_prompt() + f"Question: {item['question']}\nAnswer:"

```

A.4 Decoding configuration

Both benchmarks use a maximum of 1,024 generated tokens per trajectory. Temperature-sampling baselines enable stochastic sampling and use the temperatures in Table 4.1. FLEET disables stochastic sampling, decodes greedily, and receives the generator’s raw logits before any temperature scaling. Trajectories that reach the token limit are submitted to the corresponding verifier or ORM in their unfinished form.